GOLD: File I/O (Wed Oct 7)

Announcements

Midterm 1 Retakes

Retakes are TONIGHT in the 6:30 - 7:30 pm or 8:00 - 9:00 pm window. **Seat assignments are in the Brightspace gradebook.**

Office Hours

Prof. Borkowski's will not have Thursday office hours tomorrow.

Fall Break: There are no labs or office hours Monday October 12 or Tuesday October 13.

Pattern Matching (from Monday)

We keep seeing Java code that asks "*is this thing of that kind?*" and, when the answer is yes, gives it a name:

where	**syntax**	**the question it asks**
in `equals` (and see the Casting slide)	`if (o instanceof Card c)`	is `o` a Card? Then call it `c`
after a `try`	`catch (NumberFormatException e)`	is the failure this kind? Then call it `e`
the general form	`case Coach c ->` inside a `switch`	is `p` a Coach? Then call it `c`

The general form:

```
class Main {
    String describe(Payable p) {
        return switch (p) {
            case Coach c    -> "coach " + c.getName();
            case Building b -> "a building";
            default         -> "something else that can be paid";
        };
    }

    void main() {
        Payable[] payroll = { new Coach("Knuth"), new Building("Matthews") };
        for (Payable p : payroll) {
            IO.println(describe(p) + ": " + p.weeklyPay());
        }
    }
}

interface Payable {
    String payTo();
    double weeklyPay();
}

class Person {
    private String name;
    Person(String name) { this.name = name; }
    String getName() { return name; }
}

class Coach extends Person implements Payable {
    Coach(String name) { super(name); }
    @Override public String payTo() { return getName(); }
    @Override public double weeklyPay() { return 2400.0; }
}

class Building implements Payable {
    private String name;
    Building(String name) { this.name = name; }
    @Override public String payTo() { return "Physical Facilities"; }
    @Override public double weeklyPay() { return 800.0; }
}
```

```
coach Knuth: 2400.0
a building: 800.0
```

- The cases are tried top to bottom and the first match wins, exactly like the catches.

- A broader type first is a compile error, exactly like the catches: `this case label is dominated by a preceding case label`.
- `default` is the catch-all for every kind we did not name.

What's different here?

Previously:

- the payroll loop chose the code by **override**
- the object decides,
- and a new class needs no edit to the loop.

With pattern matching:

- A switch on the type lets the **caller** decide,
- and a new operation needs no edit to the classes.

Best Practices (from Monday)

When *not* to catch

Two kinds of trouble:

	Expected failure	Bug
Examples	bad user input, an empty deck, a dropped connection	`a[i]` out of bounds, `null.length()`, negative quantity reaching `price()`
Right response	**catch:** re-prompt, default, friendly message	**let it crash:** read the trace, *fix the code*

Two habits that make debugging miserable:

- **The everything-catcher:**
  - `catch (Exception e)` around whole programs "so it stops crashing"
  - it doesn't fix anything,
  - it just stops *telling you*.

- **The empty catch block:** `catch (...) { }`
  - **Never leave a catch block empty**
  - it doesn't fix anything
  - it also just stops telling you
  - at minimum, print something!

Where Does the Data Go?

Where Does the Data Go?

Boiler Burgers closed at midnight. Before the register program ended, it wrote every sale to a file:

```
5.49
12.98
7.25
5.49
21.47
```

This is `sales.txt`. The program has ended and is gone; now the data only lives here.

Today's lecture: how we can:

- read it,
- total it up,
- and write the day's sales report to another file.

Data should persist

- Everything in your variables lives in **RAM**
- RAM comes and goes:
    - programs end (or crash)
    - systems reboot
- **Files last** (well... comparatively speaking)
- Programs save data to files to:
    - pick up tomorrow where they left off today
    - hand results to *other* programs

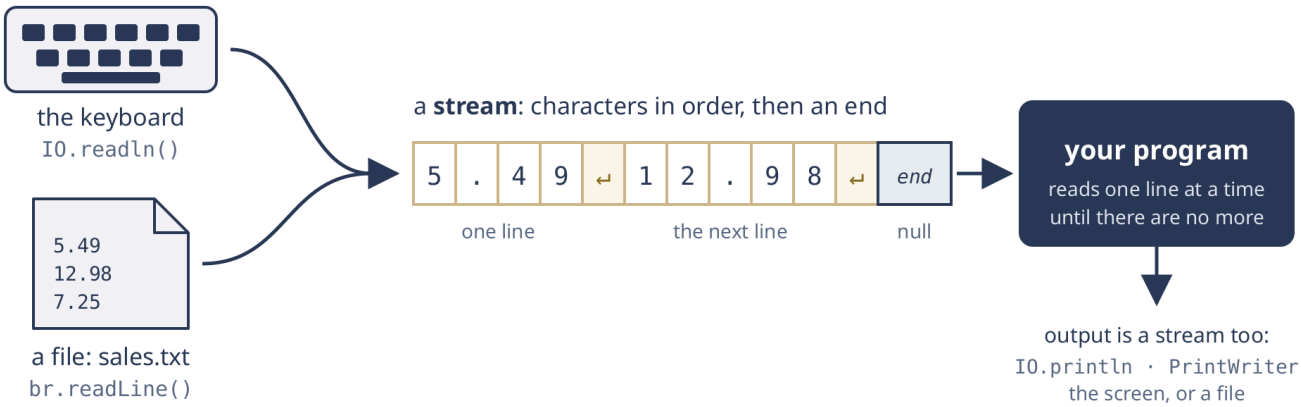
- A file is the simplest form of **persistence**: data that outlives the program

Files

A file is a stream with a name

A **stream** is a sequence of characters that arrives in order, one after another, and then ends.

Your programs have been reading one already: the keyboard, through `IO.readln`, one line at a time.



A file is the same idea with a **name**. Its characters are already waiting on disk, and the program reads them the same way, one line at a time, until there are no more.

Output is a stream too:

- `IO.println` sends characters to the screen
- Today `PrintWriter` will send them to a file.

Reading a file is the input you already know, but aimed at a name instead of the keyboard.

Unlike the keyboard, a file ends at some point instead of the programming waiting for more input to arrive.

(The shell can aim the keyboard input at a file too; see the extra reading at the end.)

Reading a File

Reading a file: the ceremony

Three steps: **open**, **read**, and **close**:

```
void main() throws IOException {
    salesFile();                                    // given: makes sales.txt so this sl

    // 1. OPEN
    BufferedReader br = new BufferedReader(new FileReader("sales.txt"));

    String line = br.readLine();                    // 2. READ one line at a time,
    while (line != null) {                          //    null means "no more file"
        IO.println(line);
        line = br.readLine();
    }
    br.close();                                     // 3. CLOSE
}

// Given. The register wrote this file during the day; the slide writes it so that Run
void salesFile() throws IOException {
    PrintWriter pw = new PrintWriter(new FileOutputStream("sales.txt"));
    pw.println("5.49");
    pw.println("12.98");
    pw.println("7.25");
    pw.println("5.49");
    pw.println("21.47");
    pw.close();
}
```

```
5.49
12.98
7.25
5.49
21.47
```

- The OPEN line.
  - `throws IOException` on `main` is our *declaration* of not catching a checked exception.
  - reading a file can fail, the failure is checked, and this method passes it to its caller.

    - The double-decker constructor `new BufferedReader(new FileReader(...))`: an efficient reader wrapped around a basic one. We write it as an idiom.

- READ one line at a time. `readLine()` hands back `null` when there is no more file: the next slide.

- CLOSE when you are done. Always do this!

- In a class of your own, as in the labs, add `import java.io.*;` at the top. Any `void main()` files import it for you.

- `salesFile()` at the bottom is given code to write a specific file.

null Is the File's Sentinel

`null` is the file's sentinel

You already know the file-reading structure. It's the usual sentinel loop:

Sentinel loop	File loop (today)
`line = IO.readln(...)`	`line = br.readLine()`
stop when `line.equals("done")`	stop when `line == null`
read again at the end of the body	read again at the end of the body

The only difference: you chose `"done"` / `-1` as the sentinel.

The file's end **is** the sentinel: `readLine()` returns `null` when there is no more file.

```
String line = br.readLine();          // read the first line
while (line != null) {                // the end of the file IS the sentinel
    IO.println(line);
    line = br.readLine();             // read again at the end of the body
}
```

Total the Sales

Total the sales

We know:

- Reading the file
- the accumulator pattern to add up data

Let's put them together:

```
void main() throws IOException {
    salesFile();                                    // given

    BufferedReader br = new BufferedReader(new FileReader("sales.txt"));
    double total = 0;
    int count = 0;

    String line = br.readLine();
    while (line != null) {
        total += Double.parseDouble(line);
        count++;
        line = br.readLine();
    }
    br.close();

    IO.println("Sales: " + count);
    IO.println(String.format("Total: $%.2f", total));
}

// Given. The register wrote this file during the day; the slide writes it so that Run
void salesFile() throws IOException {
    PrintWriter pw = new PrintWriter(new FileOutputStream("sales.txt"));
    pw.println("5.49");
    pw.println("12.98");
    pw.println("7.25");
    pw.println("5.49");
    pw.println("21.47");
    pw.close();
}
```

```
Sales: 5
Total: $52.68
```

Predict: Blank Line or End of File?

Predict: blank line vs. end of file

`probe.txt` has three lines, and the middle one is **blank**:

```
alpha

beta
```

The program calls `br.readLine()` **four** times and prints each result. What are the four values?

```
void main() throws IOException {
    BufferedReader br = new BufferedReader(new FileReader("/course/lectures/probe.txt"
    IO.println(br.readLine());
    IO.println(br.readLine());
    IO.println(br.readLine());
    IO.println(br.readLine());
    br.close();
}
```

Pause and predict (poll time)

Q. What are the four values?

- **A.** `alpha`, `beta`, `null`, `null`: the blank line is skipped
- **B.** `alpha`, an empty string, `beta`, `null`
- **C.** `alpha`, an empty string, `beta`, then a crash: reading past the end
- **D.** `alpha`, `null`, `beta`, `null`: a blank line reads as `null`

Catch Order (from Monday)

Here's the same program from last time, but with its catches reordered:

```
try {
    int n = Integer.parseInt(IO.readln("Number? "));
    IO.println(100 / n);
} catch (Exception e) {
    IO.println("something went wrong");
} catch (NumberFormatException e) {
    IO.println("that wasn't a number");
}
```

Pause and predict

Q. What happens?

- **A.** The more specific catch silently never runs
- **B.** Compile error: `exception NumberFormatException has already been caught`
- **C.** Both catches run, the broad one first
- **D.** Java reorders them automatically to run the `NumberFormatException` catch

The Missing File

Watch this fail: the file that wasn't there

Let's make a one-character typo, `"sale.txt"` for `"sales.txt"`:

```
Exception in thread "main" java.io.FileNotFoundException: sale.txt (No such file or di
    at java.base/java.io.FileInputStream.open0(Native Method)
    at java.base/java.io.FileInputStream.open(FileInputStream.java:185)
    ...
```

We're always one missing file away from a crash.

`FileNotFoundException` is a kind of `IOException`, and it is **checked.**

Now we catch it and say something useful:

```
void main() {
    try {
        BufferedReader br = new BufferedReader(new FileReader("/course/lectures/sale.t
        IO.println(br.readLine());
        br.close();
    } catch (FileNotFoundException e) {
        IO.println("No such file: " + e.getMessage());
    } catch (IOException e) {
        IO.println("Trouble reading the file: " + e.getMessage());
    }
    IO.println("Still running.");
}
```

```
No such file: sale.txt (No such file or directory)
Still running.
```

The order of the two catches is the rule from the poll we just ran:

- `FileNotFoundException` is a kind of `IOException`, so the specific one comes first.
- If we swap them the program does not compile: `exception FileNotFoundException has already been caught`.

Writing: PrintWriter

Writing: `PrintWriter`

The mirror of the reading ceremony: open, print, and close.

```
void main() throws IOException {
    int count = 5;
    double total = 52.68;

    PrintWriter pw = new PrintWriter(new FileOutputStream("report.txt"));
    pw.println("Boiler Burgers Daily Report");
    pw.println("Sales: " + count);
    pw.println(String.format("Total: $%.2f", total));
    pw.println(String.format("Average: $%.2f", total / count));
    pw.close();                                  // do NOT skip this

    IO.println("report.txt now holds:");
    showFile("report.txt");                      // given: reads the file back
}

// Given. Reads a file back and prints it: the ceremony from the first half of today.
void showFile(String path) throws IOException {
    BufferedReader br = new BufferedReader(new FileReader(path));
    String line = br.readLine();
    while (line != null) {
        IO.println(line);
        line = br.readLine();
    }
    br.close();
}
```

```
report.txt now holds:
Boiler Burgers Daily Report
Sales: 5
Total: $52.68
Average: $10.54
```

- `pw.println(...)` is `IO.println(...)`'s cousin: the same behavior, aimed at a file instead of the screen.
- Everything you know about building output lines transfers.

- The OPEN line is a double-decker `new PrintWriter(new FileOutputStream("report.txt"))`
    - the `FileOutputStream` builds its own `File` object.

- `showFile`, given at the bottom, reads the file back.

Watch This Fail: Forgot to Close

Watch this fail: forgot to close

Delete one line:

```
void main() throws IOException {
    PrintWriter pw = new PrintWriter(new FileOutputStream("report2.txt"));
    pw.println("Boiler Burgers Daily Report");
    pw.println("Sales: 5");
    // forgot pw.close();

    IO.println("report2.txt: " + new File("report2.txt").length() + " bytes");   // gi
}
```

The program runs. No error. The last line, given, prints the size of the file afterwards.

Pause and predict (poll time)

Q. What is in `report2.txt` afterwards?

- **A.** Both lines
- **B.** The first line only
- **C.** Nothing: the file is empty, 0 bytes
- **D.** Nothing: the program crashes when it ends with the file still open

finally: Cleanup That Always Runs

`finally`: cleanup that runs no matter what

A file you did not close is a file you may not have written.

Why:

- output is **buffered**
- your `println`s pile up in memory, and
- they're only pushed to disk when the buffer fills… or when you `close()`.

A crash in the middle of the loop skips `pw.close()` just as surely as forgetting it.

Java has a block for cleanup that must happen either way: one more block after a `try`, named `finally`.

```
int f() {
    try {
        return 1;
    } finally {
        IO.println("finally ran anyway");
    }
}

void main() {
    IO.println(f());
}
```

Pause and predict (poll time)

Q. Does `finally ran anyway` print, and what does `main` print?

- **A.** No: `return` leaves the method at once, and `main` prints `1`
- **B.** Yes, but only when an exception is thrown
- **C.** Yes: `finally ran anyway`, then `1`
- **D.** It does not compile: `finally` needs a `catch`

try-with-resources

Putting `close()` in `finally` works:

```
void main() throws IOException {
    BufferedReader br = null;
    try {
        br = new BufferedReader(new FileReader("sales.txt"));
        String line = br.readLine();
        while (line != null) {
            IO.println(line);
            line = br.readLine();
        }
    } finally {
        if (br != null) {
            br.close();                          // runs whether the loop finished or
        }
    }
}
```

Java has a shorter form. Declare the reader (or writer) in parentheses after `try`, and Java closes it for you at the closing brace, whether the block finished or threw:

```
void main() throws IOException {
    salesFile();                                    // given

    double total = 0;
    try (BufferedReader br = new BufferedReader(new FileReader("sales.txt"))) {
        String line = br.readLine();
        while (line != null) {
            total += Double.parseDouble(line);
            line = br.readLine();
        }
    }                                               // br.close() has happened here, no

    try (PrintWriter pw = new PrintWriter(new FileOutputStream("report.txt"))) {
        pw.println("Boiler Burgers Daily Report");
        pw.println(String.format("Total: $%.2f", total));
    }                                               // and pw.close() here

    showFile("report.txt");                         // given: reads the file back
}

// Given. Reads a file back and prints it: the ceremony from the first half of today.
void showFile(String path) throws IOException {
    BufferedReader br = new BufferedReader(new FileReader(path));
    String line = br.readLine();
    while (line != null) {
        IO.println(line);
        line = br.readLine();
    }
    br.close();
}

// Given. The register wrote this file during the day; the slide writes it so that Run
void salesFile() throws IOException {
    PrintWriter pw = new PrintWriter(new FileOutputStream("sales.txt"));
    pw.println("5.49");
    pw.println("12.98");
    pw.println("7.25");
    pw.println("5.49");
    pw.println("21.47");
    pw.close();
}
```

```
Boiler Burgers Daily Report
Total: $52.68
```

- The resource is closed at the closing brace, no matter what.

- There is no `close()` line to forget.
- This is the form to use from today on, in lecture and in the labs.
- The 1998 ceremony is for reading other people's code.
- Fine print: `close()` itself can throw `IOException`, so a try-with-resources that catches only `FileNotFoundException` does not compile.
    - Catch `IOException`, which covers both.

Build Your Own Tool: readAllLines

Build your own tool: `readAllLines`

The ceremony of reading from a file never changes:

- read each line and
- do something
- close it

Let's write it **once**, as a helper, and reuse it forever.

- We can read every line into an ArrayList

- In practice, files may be too large to read all at once
    - Then you process them line-by-line on the fly

```
void main() throws IOException {
    salesFile();                                    // given

    ArrayList<String> lines = readAllLines("sales.txt");

    double total = 0, max = 0;
    for (String s : lines) {
        double sale = Double.parseDouble(s);
        total += sale;
        if (sale > max) {
            max = sale;
        }
    }
    IO.println("Sales: " + lines.size());
    IO.println(String.format("Total: $%.2f", total));
    IO.println(String.format("Biggest sale: $%.2f", max));
}

ArrayList<String> readAllLines(String path) throws IOException {
    ArrayList<String> lines = new ArrayList<>();
    try (BufferedReader br = new BufferedReader(new FileReader(path))) {
        String line = br.readLine();
        while (line != null) {
            lines.add(line);
            line = br.readLine();
        }
    }
    return lines;
}

// Given. The register wrote this file during the day; the slide writes it so that Run
void salesFile() throws IOException {
    PrintWriter pw = new PrintWriter(new FileOutputStream("sales.txt"));
    pw.println("5.49");
    pw.println("12.98");
    pw.println("7.25");
    pw.println("5.49");
    pw.println("21.47");
    pw.close();
}
```

```
Sales: 5
Total: $52.68
Biggest sale: $21.47
```

Now every file program can start the same way:

```
ArrayList<String> lines = readAllLines("sales.txt");
```

and from there it is list work.

The Format Is a Contract

The format is a contract

The register could also log *what* was sold.

One line per order: `item,quantity,price`

```
BURGER,2,5.49
FRIES,3,2.75
SHAKE,1,4.95
```

We call this format **comma-separated values** or CSV.

```
void main() throws IOException {
    ordersFile();                                   // given

    double revenue = 0;
    for (String line : readAllLines("orders.csv")) {
        String[] parts = line.split(",");          // {"BURGER", "2", "5.49"}
        int qty = Integer.parseInt(parts[1]);
        double price = Double.parseDouble(parts[2]);
        revenue += qty * price;
    }
    IO.println(String.format("Revenue: $%.2f", revenue));
}

ArrayList<String> readAllLines(String path) throws IOException {
    ArrayList<String> lines = new ArrayList<>();
    try (BufferedReader br = new BufferedReader(new FileReader(path))) {
        String line = br.readLine();
        while (line != null) {
            lines.add(line);
            line = br.readLine();
        }
    }
    return lines;
}

// Given. One order per line: item,quantity,price.
void ordersFile() throws IOException {
    PrintWriter pw = new PrintWriter(new FileOutputStream("orders.csv"));
    pw.println("BURGER,2,5.49");
    pw.println("FRIES,3,2.75");
    pw.println("SHAKE,1,4.95");
    pw.close();
}
```

A file is just characters.

The **format** (what the columns mean, in what order) is a *promise between the program that writes and the program that reads*.

If you break the promise then your data isn't readable (at least not without a lot of human effort).

Append or Overwrite?

Append, or overwrite?

Opening a file for writing **erases it**, unless you say otherwise:

```
new FileOutputStream("log.txt")          // OVERWRITE: the file starts empty
new FileOutputStream("log.txt", true)    // APPEND: keep the contents, add to the end
```

```
void main() throws IOException {
    PrintWriter a = new PrintWriter(new FileOutputStream("log.txt"));          // OVER
    a.println("Monday: 5 sales");
    a.close();

    PrintWriter b = new PrintWriter(new FileOutputStream("log.txt", true));    // APPE
    b.println("Tuesday: 7 sales");
    b.close();
    IO.println("after the append:");
    showFile("log.txt");                                                       // give

    PrintWriter c = new PrintWriter(new FileOutputStream("log.txt"));          // OVER
    c.println("Wednesday: 6 sales");
    c.close();
    IO.println("after the overwrite:");
    showFile("log.txt");
}

// Given. Reads a file back and prints it: the ceremony from the first half of today.
void showFile(String path) throws IOException {
    BufferedReader br = new BufferedReader(new FileReader(path));
    String line = br.readLine();
    while (line != null) {
        IO.println(line);
        line = br.readLine();
    }
    br.close();
}
```

```
after the append:
Monday: 5 sales
Tuesday: 7 sales
after the overwrite:
Wednesday: 6 sales
```

The register uses append: each day adds to the log.

A report uses overwrite: you want *this* report, not every report ever.

Extra Reading: Streams from the Shell

Extra reading: not presented in lecture.

You've been using streams all along

We can run one of our keyboard I/O programs without the keyboard

For instance

```
java WordCount.java < war_and_peace.txt
```

or you can pipe characters *into* it:

```
printf '5.49\n3.50\n-1\n' | java LemonadeStand.java
```

In the Unix shell, the `|` character:

- takes the output of the program/command to the left and
- feeds/pipes it to the program/command on the right

Extra Reading: The Modern Door

Extra reading: not presented in lecture.

The modern door (read, don't write)

The `readAllLines` helper that we built earlier is now built into modern Java:

You built (and we use)	Built into `java.nio` (2011+)
`readAllLines("sales.txt")` → `ArrayList<String>`	`Files.readAllLines(Path.of("sales.txt"))` → `List<String>`
the whole `PrintWriter` ceremony	`Files.writeString(Path.of("report.txt"), text)`

You now know what's inside those one-liners.

- This course (labs, tests, reference sheets) uses the classic forms.
- If you meet `Files`/`Path` in the wild or already use them, they produce the same files.
- Test suites check your output, not your API.
- Sometimes there can be a trailing newline `\n` difference to debug
- Note: `Files` and `Path` are in `java.nio` not `java.io`

Additionally, all of the `Reader` class children have a `readAllLines()` method that returns a `List<String>` .

- be sure you know how big the file is; otherwise, you could run out of memory!

Extra Reading: Text and Binary

Extra reading: not presented in lecture.

Text files and binary files

Open your own files in the editor:

- `sales.txt`, `report.txt`, any `.java` are **text**:
  - human-readable characters;
  - what you see is what's stored

- `PrintSales.class` are **binary**:
  - bytes meant for the JVM, not for eyes;
  - the editor shows garbage

Everything today was text I/O.

Binary formats exist (`.class`, `.jpg`, `.zip`) and need their own programs to make sense of them.

Take Home: The Rain Report

A weather station logs one number per day: millimetres of rain. The log is `rain.txt`, one day per line, and it is in your workspace.

Part 1: read and compute. In `Main.java`, read every line of `rain.txt` as a `double`. You know two ways: the ceremony (open, read until `null`, close) or the `readAllLines` helper from lecture. Count the days, total the rain, and find the wettest day.

Part 2: write the report. Write `rain-report.txt` with exactly these four lines, in this order, for the data as given:

```
Days: 7
Total: 23.4 mm
Average: 3.34 mm
Wettest: 8.1 mm
```

Format the total and the wettest day with `%.1f` and the average with `%.2f` (`String.format`), so that floating-point noise such as `23.400000000000002` never reaches the report.

Part 3: the missing file. If `rain.txt` cannot be opened, print one line, `Cannot open rain.txt:` followed by the exception's message, and stop without writing a report. Try it: rename `rain.txt` in the workspace and press **Run**.

Your tests check the report file line by line and the missing-file message, not which classes you used. Press **Test** to run them; they put `rain.txt` back when they finish.